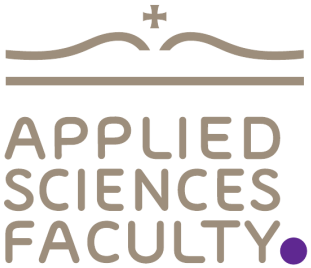
C++ — лекція 15. Поліморфізм





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- Часто потрібно, щоб всі класи певної ієрархії успадкувань мали спільний інтерфейс — набір методів, що змушують[1] об'єкти виконати певні дії.
- ◇ При чому, об'єкти різних класів можуть виконувати ці дії по різному.
- Така можливість називається **поліморфізмом**.
- Один із найпоширеніших підходів, до її реалізації — віртуальні функції.

[1] В іншій термінологічній традиції — передають об'єкту повідомлення.

Ілюстрація проблеми

```
class Base1{
public:
    void print(){
        cout << "This is Base1" << endl;
    }
};

class DerivedA1: public Base1{
public:
    void print(){
        cout << "This is DerivedA1" << endl;
    }
};

class DerivedB1: public Base1{
public:
    void print(){
        cout << "This is DerivedB1" << endl;
    }
};
```

```
DerivedA1 a1;
DerivedB1 b1;
a1.print();
b1.print();

cout << "--------" << endl;

// Завдяки принципу підстановки
// можемо зберігати вказівни-
ки на нащадків
// у вказівниках на базовий клас
Base1 *p1[2];
p1[0]=&a1;
p1[1]=&b1;
for(int i = 0; i < 2; ++i){
    p1[i]->print();
}
// Однак результат ...
```

Результат:

```
This is DerivedA1
This is DerivedB1
-------
This is Base1
This is Base1
```

Це — так зване раннє або статичне зв'язування. Воно відбувається під час компіляції.

Реалізація для користувача надзвичайно проста:

```
class Base2{
public:
    virtual void print(){
        cout << "This is Base2" << endl;
    }
};

class DerivedA2: public Base2{
public:
    void print() override {
        cout << "This is DerivedA2" << endl;
    }
};

class DerivedB2: public Base2{
public:
    virtual void print() override {
        cout << "This is DerivedB2" << endl;
    }
};
```

Нам же потрібен наступний результат:

```
DerivedA2 a2;
DerivedB2 b2;
a2.print();
b2.print();
cout << endl;

Base2 *p2[2];
p2[0]=&a2;
p2[1]=&b2;
for(int i = 0; i < 2; ++i){
    p2[i]->print();
}
```

Результат

```
This is DerivedA2
This is DerivedB2

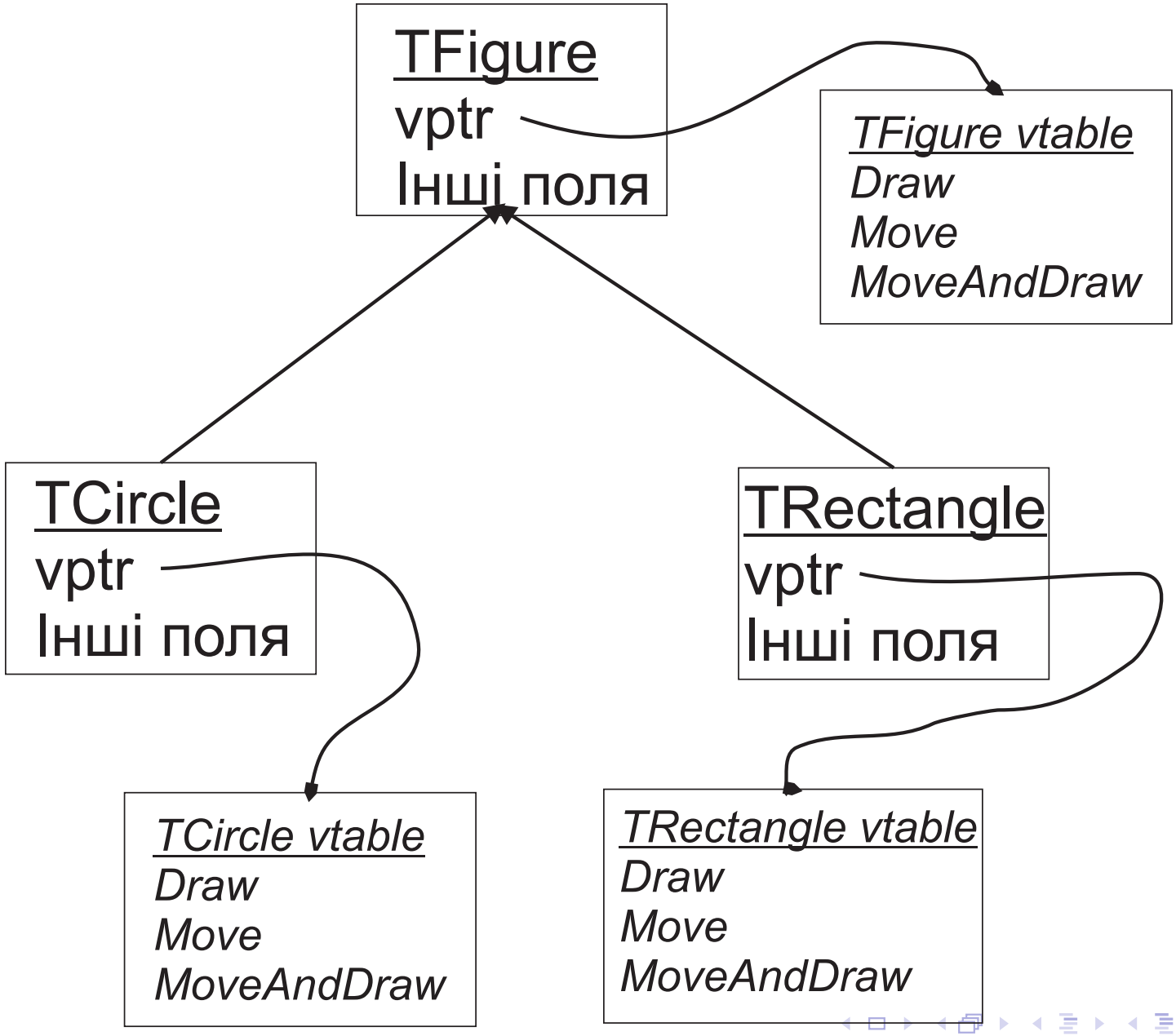
This is DerivedA2
This is DerivedB2
```

Як працюють віртуальні функції

- Клас, який має хоча б одну віртуальну функцію, має приховане поле, умовно назвемо його vptr, яке є вказівником на таблицю віртуальних функцій, умовно — vtable[2].
- Кожній віртуальній функції присвоюється номер в цій таблиці.
- Виклик віртуальної функції виглядає так:
  - в об'єкті знаходиться вказівник vptr,
  - за його допомогою знаходиться таблиця віртуальних функцій.
  - З vtable дістається вказівник із номером, що відповідає шуканій віртуальній функції.
  - За цим вказівником здійснюється виклик віртуальної функції.
- Таким чином, так як кожен клас-нащадок має **свою** таблицю віртуальних функцій, навіть якщо виклик здійснюється із допомогою вказівника на базовий клас, викликатиметься правильна функція.

[2] Тобто, vtable є особливим статичним членом класу. Особливим, оскільки нащадки мають свою, на відміну від звичайних статичних членів даних

Ілюстрація


TFigure
vptr
Інші поля
TFigure vtable
Draw
Move
MoveAndDraw
TCircle
vptr
Інші поля
TRectangle
vptr
Інші поля
TCircle vtable
Draw
Move
MoveAndDraw
TRectangle vtable
Draw
Move
MoveAndDraw

Ілюстрація низькорівнева – не віртуальна функція

- Розглянемо виклик функції у такому (невірному!) коді:

```
class B{
public:
        void fn();
};

class D: public B{
public:
        void fn();
};

B* ptr = new D{};
ptr->fn();                                  // 1
```

- Тоді, рядок 1 на асемблері x86 виглядатиме, спрощено, так:

```
mov  rdi, [ptr] ; Put object address to the RDI
                ; -- first function argument for the
                ; x86 calling convention
call B::fn      ; B::fn here means the address
                ; of the function
```

Ілюстрація низькорівнева – віртуальна функція

- Розглянемо виклик функції у такому (невірному!) коді:

```
class B{
public:
        virtual void fn();
};

class D: public B{
public:
        void fn() override;
};

B* ptr = new D{};
ptr->fn();                                  // 1
```

- Тоді, рядок 1 на асемблері x86 виглядатиме, спрощено, так:

```
mov  rdi, [ptr] ;
mov  rax, [rdi] ; vptr is the first member of object here
call rax        ; call the first item in the vtable
```

- Побачити код вище в асемблерному виводі не так просто – він або буде занадто складним, або компілятор постарається здійснити девіртуалізацію.

Заміщення (override) віртуальних функцій

- Оголошуються віртуальні функції за допомогою ключового слова **virtual**:

```
virtual void print ();
```

- Якщо в класі-нащадку оголошено функцію із таким же набором параметрів, і тим же типом, що повертається[3], вона теж буде віртуальною, і буде заміщати (override) відповідну функцію з базового класу.
- **УВАГА!** Функція із тим же іменем, але іншими аргументами, не буде заміщати відповідну віртуальну функцію з базового!
  - ◇ Відбуватиметься звичайне статичне перевизначення.
  - ◇ Або буде створено нову віртуальну функцію, якщо перед оголошенням вказати virtual.
- Для запобігання цій проблемі, слід після оголошення функції, яка має замістити предкову віртуальну функцію, писати **override**[4] – тоді, у випадку помилки у вказанні аргументів, замість створення не пов'язаної функції, відбудеться помилка компіляції:

```
void print () override ;
```

  - ◇ Під час заміщення віртуальних функцій, ключове слово virtual дозволене, але не обов'язкове, і, певне, небажане.

[3] За одним винятком, див. далі.

[4] Необхідність так робити – наслідок зворотної сумісності. Якби мова проектувалася зараз, ймовірно, override був би за замовчуванням, але був модифікатор "*nooverride*" для екзотичних випадків.

Статичне зв'язування віртуальні функції в конструкторах і деструкторах

- **УВАГА!** Віртуальні функції в конструкторах та деструкторах зв'язуються статично!
- ◇ Поліморфізму в них не буде – це природно, адже під час роботи деструктора чи конструктора, "нащадкової" частини класу вже чи ще не існує.
- Однак, в інших методах зв'язування все рівно динамічне, зокрема – в функціях, що реалізовані в базових класах. Це дуже важлива властивість.

Список аргументів віртуальних функцій має збігатися!

```
class Base{
public:
    virtual void fn(int x){ cout << "Base fn int" << endl; };
};

class Derived: public Base{
public:
    void fn(double x){ cout << "Derived fn double" << endl; };
    void fn(int x){ cout << "Derived fn int" << endl; };
};
```

```
.....
Derived d;
Base *bp = &d;
d.fn(1);        // Derived fn int
d.fn(1.2);      // Derived fn double
bp->fn(1);      // Derived fn int
bp->fn(1.2);    // Derived fn int <==!
```

Віртуальні деструктори

- УВАГА! Якщо в класі є хоча б одна віртуальна функція, варто робити і деструктор віртуальним.
- Якщо хоча б в одному із нащадків конструктор може виконувати нетривіальні операції[5], варто потурбуватися, щоб для об'єктів викликався коректний деструктор.
- Невеликий виняток – не віртуальний, захищений деструктор, лише для використання нащадками: "Virtuality" by H. Sutter, Guideline #4.
- Ще один екзотичний і небезпечний випадок: "A virtual destructor in C++, when?" – коли об'єкти не видаляються поліморфно і є важливим мінімізувати їх розмір.

[5] А гарантувати, що такого не буде, майже неможливо! Перевага ООП, зокрема, в тому, що нащадків можна додавати потім.

Віртуальні "конструктори"

- **Віртуальні конструктори існувати не можуть.** Щоб створити об'єкт, компілятор повинен знати його тип.
- Однак, бувають випадки, коли від конструктора хочеться отримати поліморфну поведінку.
  - Перший випадок — "маю вказівник на тип базового класу, хочу повну копію об'єкту, на яку він вказує[6]",
  - другий — маючи вказівник, "хочу новий об'єкт того ж типу, що має об'єкт, на який вказує вказівник".
- Схожа ідея – реєстрація класу у фабриці об'єктів: `https://www.cppstories.com/2023/ub-factory-constinit/`.

[6] Тобто — без зрізки

Віртуальні "конструктори" – приклад

```
class BaseClass{
    .....
    BaseClass();              // Конструктор за замовчуванням
    BaseClass(const BaseClass&); // Конструктор копіювання

    virtual BaseClass* create_the_same(){
        return new SomeClass();
    }
    virtual BaseClass* clone(){
        return new SomeClass(*this);
    }
};

class DerivedClass: public BaseClass{
    .....
    DerivedClass();           // Конструктор за замовчуванням
    DerivedClass(const DerivedClass&); // Конструктор копіювання

    DerivedClass* create_the_same() override {
        return new DerivedClass();
    }
    DerivedClass* clone() override {
        return new DerivedClass(*this);
    }
```

Коваріантність типів, що повертаються з віртуальних ф-цій

- Щоб такі функції можна було заміщати, існує пом'якшення вимоги того, щоб прототипи віртуальних функцій збігалися: якщо вихідна віртуальна функція повертає вказівник на тип B, в потомках вона може повертати інший тип, D, за умови, що B є його відкритим базовим класом (Згадуємо також принцип підстановки).
- Така властивість називається коваріантністю типів значень, що повертаються – можна повертати більш конкретний тип, ніж в предку. Див. також `https://en.wikipedia.org/wiki/Covariance_and_contravariance_(computer_science)`.

```
class A {};
class B : public A{};

struct Base{
    virtual A* fn(){ return new A(); }
};
struct Derived: public Base{
    virtual B* fn(){ return new B(); }
};
```

- Детальніший аналіз (та майбутнє із метакласами і рефлексіями) див. `https://herbsutter.com/2019/10/03/gotw-ish-solution-the-clonable-pattern/`.
- Ремарка – контраваріантність, це поняття із ООП, яке означає можливість методу приймати більш загальний тип, ніж він приймав у предка. C++ чи Java **не підтримується**.

Ціна віртуальних функцій

- Виклик віртуальних функцій обходиться суттєво дорожче, через додаткове звертання до пам'яті і виклик функції за вказівником.
  - ◇ Для сучасних процесорів із довгими конвеєрами і те і те є непростим. Тому останні десятиліття процесори оптимізуються саме під виконання віртуальних функцій.
  - ◇ Див., зокрема: "The true price of virtual functions in C++".
- Поява першої віртуальної функції збільшує розмір об'єкту на vptr. Іноді це може бути дуже суттєво.

Абстрактні класи

- Віртуальні функції задають інтерфейс — набір "команд", на які повинні реагувати всі потомки. Однак, не завжди базовий клас може дати осмислену їх реалізацію.
- Для явного вираження цього факту можна створювати чисто віртуальні функції:

```
class SomeClass{
    .....
    virtual void function(int x) = 0;
};
```

- Не можна створювати об'єкти класів із чисто віртуальними функціями! Ця заборона змушує потомків таки реалізовувати весь необхідний інтерфейс.
- Класи із чисто віртуальними функціями називають **абстрактними**.

Реалізація чисто віртуальних функцій

Однак, можливим є створення реалізації чисто віртуальних функцій. Тоді клас-нащадок все рівно мусить замістити таку функцію, однак, за потреби, зможе скористатися визначенням за замовчуванням.

```
class SomeBaseClass{
    .....
    virtual void function(int x) = 0;
};

void SomeBaseClass::function(int x)
{ ..... }

class SomeDerivedClass: public SomeBaseClass{
    .....
    virtual void function(int x){
        SomeBaseClass::function(x);
    }
};
```

Що таке добре і що таке погано

Щоб правильно зорієнтуватися у просторі можливостей, слід користуватися наступними рекомендаціями:

- Не-віртуальна функція — інваріант. Тобто, вона фіксує поведінку для всіх потомків класу, в якому вона оголошена.
- **Слід уникати перевизначення невіртуальних функцій!** Поведінка таких функцій залежатиме від того, яким чином їх викликають, зокрема, від типу вказівника, через який їх було викликано.
- Чисто віртуальна функція оголошує (”обіцяє”) інтерфейс, який буде у всіх потомків.
- Віртуальна функція задає як інтерфейс, так і реалізацію за замовчуванням.
- Чисто віртуальні функції із реалізацією – своєрідний гібрид. Вони змушують кожного потомка реалізовувати відповідний інтерфейс самостійно, однак дають готовий варіант для тих потомків, яких він влаштує.

Що таке добре і що таке погано — 2

- Варто робити віртуальні функції не-відкритими, а відкриті – не віртуальними[7]. (Ця рекомендація не стосується деструкторів!).
- Справа в тому, що відкрита віртуальна функція задає і інтерфейс і реалізацію. Іноді ці, суттєво різні, задачі можуть конфліктувати. Розділивши їх, ми залишаємо можливість налаштовувати (шляхом заміщення) поведінку потомків, при тому можемо вибирати інтерфейс так, як зручно[8].

[7] Це так званий шаблон проектування Non-Virtual Interface, NVI

[8] Наприклад, така відкрита функція може викликати декілька віртуальних, щоб в потомків була можливість змінювати лише частину поведінки.

Контроль заміщення віртуальних функцій — C++11

- Заміщаючи віртуальну функцію, легко допустити помилку, використавши інші типи аргументів. Ключове слово virtual тут не рятує — виникне ще одна віртуальна функція, з тим же іменем, яка "затінить" собою свою тезку із базового класу. Також, зміна типів параметрів віртуальних функцій (наприклад, в новій версії бібліотеки), навіть незначні, вимагатиме величезної акуратності.
- В C++11 з'явився спосіб явно виразити намір здійснити заміщення віртуальної функції — override. Якщо в предкові такої функції немає, буде помилка компіляції:

```
class Base2{
public:
    virtual A* fn() { return new A(); }
    virtual A* fn(int x) { return new A(); }
};

class Derived2: public Base2{
public:
    virtual A* fn() override { return new B(); }
    // virtual Base2* fn() override { return new B(); } Error!
    // virtual A* fn(unsigned int) override { return new B(); } Error!
};
```

Контроль заміщення віртуальних функцій 2 — C++11

- Override[9] НЕ Є частиною типу методу, тому вказувати його у визначенні не можна:

```
B* Derived2::fn(int) { return new B(); }
// B* Derived2::fn(int) override { return new B(); } Error!
```

- Іноді, хоч і рідко, може бути потреба заборонити користувачам бібліотеки класів заміщати віртуальні функції в подальшому (тобто, сама бібліотека їх заміщає, але користувачам це або не потрібно, або буде занадто складно для них реалізувати її коректно, і т.д.). Зробити це можна ще одним контекстним ключовим словом, final.
  - ◇ Одна із причин використання – допомогти компілятору виконувати девіртуалізаці, позбуватися викликів віртуальних функцій через vtable там, де відомо конкретний тип об'єкту.
- Аналогічно, за його допомогою можна заборонити успадковувати класи.
- Приклади — на наступному слайді.

[9] До речі, це ще й не ключове слово, воно належить до класу так званих контекстних ключових слів, (contextual keyword), має спеціальне значення в кількох конкретних місцях, а всюди решта може використовуватися для ідентифікаторів. Не зловживайте тим! Це зроблено лише для зворотної сумісності.

Приклад final — C++11

```
class V{
    virtual int fn() { return -1; };
};
class X: public V{
    int fn() override { return 0; };
};
class Y: public X{
    int fn() override { return 1; };
};
class Z: public Y{
    int fn() override final { return 2; };
};

class Err: public Z{
    // int fn() override { return 111; }; Error!
};

class K{};
class L : public K{};
class M final : public L{};
// class N : public M{}; // Error!
```

Підсумок

- Хороша практика:
  - вживати слово virtual, лише створюючи нову віртуальну функцію (першу в ієрархії),
  - для всіх функцій, що її заміщають – вживати або override, або final.
- Див. також `https://quuxplusone.github.io/blog/2021/02/19/virtual-final-silences-override-warning/`.
- `https://16bpp.net/blog/post/the-performance-impact-of-cpp-final-keyword/` – ілюстрація, що final може не тільки прискорювати, але й сповільнювати код.
- "When can the C++ compiler devirtualize a call?"
- "Using final in C++ to improve performance"
- "Optimising Compiler Performance: A Case For Devirtualisation"

Подвійна диспетчеризація (DRAFT)

- Іноді потрібно мати функцію, віртуальну за двома або більше об'єктами.
  - Наприклад, такі функції можуть описувати взаємодію двох об'єктів різних типів.
- На жаль, мова безпосередньо таких можливостей не підтримує.
- Вирішити можна:
  - за допомогою патерна **візитор**,
  - явного використання перетворення типів dynamic_cast та if по всіх типах,
  - або таблиць вказівників на члени.

Класична задача — рисування фігур на екрані

- Будь-яка фігура повинна вміти малювати себе.
- Фігура повинна вміти переміщатися.

Реалізовано наступні фігури.

- Прямокутник.
- Коло

Від прямокутника успадковують:

- Прямокутник із текстом.
- Контейнер для фігур із рамкою

Допоміжний код:

DRAFT! Дивіться код прикладу `l15_figures`.

- Функції масштабування (літери, які в нас служать пікселями, не квадратні):

```
int scale_x(double x){ return static_cast<int>(x); }
int scale_y(double y){ return std::round(y * scale_factor); }
```

Фігура — абстрактний клас

```
class AbstractFigure
{
public:
    AbstractFigure()              = default;
    virtual ~AbstractFigure()     = default;

    virtual void Draw(char symb='*') const = 0;
    virtual AbstractFigure& Move(int dy, int dx) = 0;

    void MoveAndDraw(int dx, int dy, char symb='*') {
        Draw(' ');
        Move(dx,dy);
        Draw(symb);
    }
};
```

Прямокутник

```
class SimpleRectangle: public AbstractFigure
{
    int y0, x0, height, width;
public:
    SimpleRectangle(int iny0, int inx0, int inh, int inw):
        y0(iny0), x0(inx0), height(inh), width(inw)
    {};

    void Draw(char symb) const override;

    SimpleRectangle& Move(int dy, int dx) override;

    int get_x1(){ return x1;};
        .....

};
```

Прямокутник – реалізація

```
void SimpleRectangle::Draw(char symb) const {
    for(int i=scale_x(x0); i<=scale_x(x0)+scale_x(width); ++i) {
        mvaddch( scale_y(y0),                    i, symb);
        mvaddch( scale_y(y0)+scale_y(height), i, symb);
    }
    for(int i=scale_y(y0); i<=scale_y(y0)+scale_y(height); ++i) {
        mvaddch(i,scale_x(x0)                    , symb);
        mvaddch(i,scale_x(x0)+scale_x(width), symb);
    }
    refresh ();
}

SimpleRectangle& SimpleRectangle::Move(int dy, int dx) {
    x0+=dx;
    y0+=dy;
    return *this;
}
```

Коло

```
class SimpleCircle : public AbstractFigure
{
    int R, yc, xc;
public:
    SimpleCircle( int iny, int inx, int inR ):
        R(inR), yc(iny), xc(inx)
    {};

    void Draw(char symb) const override;
    SimpleCircle& Move(int dy, int dx) override;
        ..............
}
```

Коло — реалізація

```
void TCircle::Draw(const char symb='*')
void SimpleCircle::Draw(char symb) const
{
    int dots = 360; //! Точок забагато, зате акуратніше коло виходить
    double fullCircle = 360;
    for(double angle=0; angle<fullCircle; angle+=fullCircle/dots) {
        double radians = 2* M_PI / fullCircle * angle;
        int curx = scale_x(R * cos( radians )) + scale_x(xc);
        int cury = scale_y(R * sin( radians )) + scale_y(yc);
        mvaddch( (cury), (curx), symb);
    }
    refresh ();
}

SimpleCircle& SimpleCircle::Move(int dy, int dx) {
    xc+=dx;
    yc+=dy;
    return *this;
}
```

Прямокутник з текстом

```
class SimpleTextRectangle: public SimpleRectangle{
    std :: string  text ;
public:
    SimpleTextRectangle(std :: string_view  intstr , int iny0, int inx0, int inh, int inw):
        SimpleRectangle(iny0, inx0, inh, inw), text( intstr )
    {}

    void Draw(char symb) const override;
        .....................
};

void SimpleTextRectangle::Draw(char symb) const {
    SimpleRectangle::Draw(symb);
    int str_pos_y=scale_y( get_y0() ) + scale_y( get_height() )/2 ;
    int str_pos_x=scale_x( get_x0() + 2);
    int str_last_x=scale_x(get_x0() + get_width() - 1);
    for(int i=0; i<str_last_x-str_pos_x && i<static_cast<int>(text.length()); ++i) {
        if (symb!=' ')
            mvaddch(str_pos_y, str_pos_x+i,text[i]); //-V107
        else
            mvaddch(str_pos_y, str_pos_x+i,symb); //-V107
    }
    refresh ();
}
```

Контейнер

```
class SimpleContainer : public SimpleRectangle {
    size_t last_figure;
    size_t max_figures;
    AbstractFigure** figures_list;
    // Не визначаючи цього оператора, ми забороняємо
    // присвоєння та копіювання.
    SimpleContainer& operator=(const SimpleContainer& rhs) = delete;
    SimpleContainer(const SimpleContainer& rhs) = delete;
public:
    SimpleContainer(int in_max_figs, int iny0, int inx0, int inh, int inw):
        SimpleRectangle(iny0, inx0, inh, inw),
        last_figure (0), max_figures(in_max_figs),
        figures_list ( new AbstractFigure*[max_figures] )
    {}

    ~SimpleContainer() override { delete[] figures_list; }

    void Draw(char symb='*') const override;
    SimpleContainer& Move(int dy, int dx) override;

    SimpleContainer& add_figure(AbstractFigure* in);
```

Реалізація контейнера — 1

```
SimpleContainer& SimpleContainer::add_figure(AbstractFigure* in) {
    if ( last_figure <max_figures) {
        in->Move( scale_x(get_y0()), scale_y(get_x0()) );
         figures_list [ last_figure ]=in;
        ++last_figure;
        return *this;
    }else{
        std :: stringstream  err_msg_s;
        err_msg_s << "Too many figures. Maximum " << max_figures << "
        throw std::range_error( err_msg_s.str() );
    }
}
```

Реалізація контейнера — 2

```
void SimpleContainer::Draw(char symb) const
{
    SimpleRectangle::Draw(symb);
    for(size_t i=0;i<last_figure;++i )
    {
        figures_list [i]->Draw(symb);
    }
}

SimpleContainer& SimpleContainer::Move(int dy, int dx)
{
    SimpleRectangle::Move(dy, dx);
    for(size_t i=0;i<last_figure;++i )
    {
        figures_list [i]->Move(dy,dx);
    }
    return *this;
}
```

Приклад роботи з фігурами — 1

```
SimpleRectangle rect1a (5,10,2,15);
rect1a.Draw();
for(int i=0;i<20;++i)
{
    delay_output(100);
    rect1a.MoveAndDraw(1,0);
}

SimpleCircle circ1a (5,7,7);
circ1a.Draw();

SimpleTextRectangle text1("Some text 1234567890!",3,10,5,22);
text1.Draw();
```

Приклад роботи з фігурами — 2

```
SimpleRectangle rect1 (0,3,0,4),   rect2 (10,12,2,8);
SimpleTextRectangle rectT1("Text!" ,10,15,10,25);
 SimpleCircle  circ1 (5,7,25);

SimpleContainer container (10,2,22,2,42);
container.add_figure(&rect1); container.add_figure(&rect2);
container.add_figure(&rectT1); container.add_figure(&circ1);
container.Draw();

int  displ=1;
while(true){
    for(int i=0;i<20;++i){
        container.MoveAndDraw(0,displ);
        delay_output(10);
    }
    char key = getch();
    if(key=='x'){
        break;
    }else{
        displ=-displ;
    }
```

Множинне успадкування

- Іноді певна сутність є розширенням двох різних понять. Наприклад, кажан є одночасно ссавцем і істотою, що вміє літати. Впихати їх в одну ієрархію може сильно ускладнити її. Тому існує так зване множинне успадкування, коли клас-нащадок успадковується від декількох базових класів.
- Таке успадкування — потужний, однак дуже небезпечний інструмент. Деякі об'єктно-орієнтовані мови взагалі відмовилися від нього. Загальне правило — можете без нього обійтися, краще обійдіться.
- Безпечним є його частковий випадок — множинне успадкування від чисто абстрактних класів без своїх даних. Власне, у такій урізаній формі воно присутнє в Java — додатково можна успадковувати лише інтерфейси, вимоги щодо протоколу — необхідних методів.
- Віртуальні базові класи (розглядаються далі), ускладнюють код та псують його структуру настільки, що їх варто максимально уникати.

- Синтаксично множинне успадкування можна виразити так:

```
class AbstractContainer
 {....};
class SimpleRectangle
 {....};
class RectangleContainerMultipleInh : public AbstractContainer,
                                      public SimpleRectangle
 {....};
```

- Об'єкти такого типу можна передавати і функціям, що очікують AbstractContainer, і функціям, що очікують SimpleRectangle:

```
void NeedRectanIge(SimpleRectangle* r){ cout << r << endl; }

void NeedContainer(AbstractContainer* c){ cout << c << endl; }
```

- Компілятор потурбується, щоб кожна функція бачила правильну частину об'єкта:

```
RectangleContainerMultipleInh rc(10, 10,10,10,10);
NeedRectanIge(&rc);
NeedContainer(&rc);
```

дасть щось типу:

```
0x22fc80
0x22fc60
```

- В прикладах, таких, як вище, часто можна використати композицію — успадкувати від одного класу, та створити поле типу іншого.
- ◇ Однак, якщо обидва класи містять віртуальні функції (нагадаємо — точки налаштування), які слід перевантажувати у нащадках, такий підхід буде дуже незручним.
- В інших випадках, хоча множинне успадкування дозволяє не писати багато функцій, що перенаправляють виклики об'єкту-члену[10], зазвичай воно привносить достатньо складностей, щоб не окупатися.

[10] Як цікавий приклад, див. клас QTextEdit із Qt, та його відносини із QTextDocument, QCursor і т.д.

Уточнення належності

- Класи-предки можуть мати методи з однаковими іменами[11]. Щоб вказати, котрий саме потрібен, можна зробити так:

```
rc.AbstractContainer::Draw('*');
rc.SimpleRectangle::Draw('*');
```

однак, краще створити у нащадку свій відповідний, який потурбується про потрібні виклики!

```
void RectangleContainerMultipleInh::Draw(char symb) const
{
    AbstractContainer::Draw(symb);
    SimpleRectangle::Draw(symb);
}
```

Тоді користувачам буде зручніше користуватися і значно важче зробити щось неправильно:

```
rc.Draw('*');
```

- Як і з одиночним успадкуванням, перевизначена функція заміщає всі функції із таким іменем із предків.

[11] Будемо вважати, що такі жахи, як відкриті дані в предках не трапляються. :-)

"Прокрустові інтерфейси"

- Оскільки прототипи (та й імена) схожих сутностей із різних базових класів можуть відрізнятися, іноді доводиться писати свої проміжні класи, які обгортають відповідні базові класи, надаючи нам інтерфейс для множинного перевантаження:

```
// Бібліотека 1
class CoolClass {.....};
// Бібліотека 2
class LibraryClass {.....};

// Наш код:
class OurLayerCoolClass: public CoolClass {.....};
class OurLayerLibraryClass: public LibraryClass {.....};

class OurWorkerClass: public OurLayerCoolClass,
                      public OurLayerLibraryClass
  {.....};
```

- Той же трюк використовується в протилежній ситуації, коли два методи різних класів мають однакову назву, чи й прототип, і абсолютно різну семантику.

Повторення базового класу

- Буває ситуація, коли обидва предки мають один і той же базовий клас.

```
class Base {.....};

class DerivedFirst : public Base {.....};
class DerivedSecond: public Base {.....};

class MultiDerived: public DerivedFirst , public DerivedSecond
  {.....};
```

  Тоді об'єкт типу MultiDerived міститиме дві копії Base — одну через DerivedFirst, DerivedFirst::Base, іншу через DerivedSecond, DerivedSecond::Base.
- Іноді така ситуація нормальна. Наприклад, класи, які можна зберігати на диск, повинні реалізувати інтерфейс Serializable, будучи нащадком відповідного класу. Тоді особиста копія у кожного із них справді може бути потрібна (хоча робота з ними і вимагає підвищеної акуратності!).
- З іншого боку, якщо ця копія містить дані, наприклад — ім'я файлу, куди зберігати, краще щоб вона була єдиною. Щоб добитися такого ефекту, слід використовувати віртуальні базові класи.

Віртуальні базові класи

- Синтаксично це виглядає так:

```
class Base {.....};

class DerivedFirst : public virtual Base {.....};
class DerivedSecond: public virtual Base {.....};

class MultiDerived: public DerivedFirst, public DerivedSecond
  {.....};
```

- Базовий клас без даних (інтерфейс) можна успадковувати в довільній кількості копій, проблеми це породжує рідко.
- Коли базовий клас містить дані, але кожна його копія потребує свої дані – ситуація теж майже безпечна, хоч і підозріла.
- Єдина копія базового класу потрібна, коли всі його нащадки повинні мати спільний доступ до одних і тих же даних.
  - ◇ В принципі, тут можна обійтися і без успадкування — зберігати дані глобально (зовсім погано), покласти їх у базовий клас, скориставшись одиночним успадкуванням (майже те саме, що і глобальні дані), зберегти окремо, в об'єктах, маючи вказівник на них.
  - ◇◇ Останній випадок цілком прийнятний, однак, згідно Страуструпа, його реалізація приводить до "ручного" відтворення того, що відбувається під час множинного успадкування. Тому, якщо така потреба є, краще скористатися віртуальними базовими класами.
- Однак, використання віртуальних базових класів вимагає нетривіального обслуговування.

Конструювання віртуальних базових класів — 1

- Конструктор базового класу має викликатися рівно раз.
- Для досягнення цього, конструктор кожного нащадка повинен викликати конструктор віртуального базового класу явно:

```
class Base{ ..................
    Base(int x, int y ){.....};
};
class DerivedFirst : public virtual Base { ..................
    DerivedFirst (): Base(1, 1) {.....}
};
class DerivedSecond: public virtual Base { ..................
    DerivedSecond(int x): Base(x, 1) {.....}
};
class MultiDerived: public DerivedFirst , public DerivedSecond{ .....
    MultiDerived (): DerivedSecond(11), Base(42, 24) {.......}
};
```

Виклик це буде відбуватися першим — як і має бути! Аж після нього йтимуть виклики конструкторів нащадків, DerivedFirst і DerivedSecond.

- Зауважте, що в результаті вони обоє будуть користуватися базовим класом, ініціалізованим як Base(42, 24)!

Конструювання віртуальних базових класів — 2

- Фактично, вважається, що кожен нащадок є прямим нащадком всіх своїх віртуальних базових класів, включаючи "непрямі".
- Для спрощення вважається, що успадковані конструктори (C++11) ініціалізують такі віртуальні базові класи — щоб не було потреби писати свій конструктор, тотожній успадкованому.
- Та ж проблема буде із конструкторами копіювання – потрібно явно копіювати базовий клас.
- Та ж проблема буде із оператором присвоєння.
- При тому, такий код двічі скопіює Base:

```
MultiDerived &MultiDerived::operator=(const MultiDerived &rhs)
{
  if (this != &rhs) {
    DerivedFirst ::operator=(*this);
    DerivedSecond::operator=(*this);
     ....
  }
  return *this;
}
```

Конструювання віртуальних базових класів — 3

- Вирішення подвійного копіювання віртуального базового класу – мати в кожному із предків (захищений) метод часткового копіювання – який копіює лише все, що не стосується базового класу:

```
MultiDerived &MultiDerived::operator=(const MultiDerived &rhs)
{
  if (this != &rhs) {
    Base::operator=(*this);
    DerivedFirst ::copy_only_self(*this);
    DerivedSecond::copy_only_self(*this);
     ....
  }
  return *this;
}
```

Виклик методів базового класу лише раз

- Узагальнюючи проблему із попередніх слайдів, віртуальний базовий може мати методи, які слід викликати в нащадках лише раз.
- Оскільки кожен прямий нащадок не може знати, як він використовуватиметься в майбутньому, він змушений викликати цю функцію.
- На жаль, елегантного рішення цієї проблеми не існує — кожен нащадок, що успадковує такий віртуальний базовий клас, повинен явно прописувати відповідні виклики.

```
class Base{ ..................
    virtual void Draw (){.....};
};
class DerivedFirst : public virtual Base { ..................
    void OwnDraw(){.....};
    void Draw(){
        Base::Draw();
        OwnDraw();
    }
};
class DerivedSecond: public virtual Base { ..................
    void OwnDraw(){.....};
    void Draw(){
        Base::Draw();
        OwnDraw();
    }
};
class MultiDerived: public DerivedFirst , public DerivedSecond{ ..........
    void OwnDraw(){.....};
    void Draw(){
        Base::Draw();
        DerivedFirst ::OwnDraw();
        DerivedSecond::OwnDraw();
        OwnDraw()
    }
};
```

- Якщо два віртуальних нащадки базового класу заміщають одну і ту ж віртуальну функцію, клас, що успадковує від них обох, змушений заміщати її — інакше виникне неоднозначність.

```
struct Base
{
    virtual void fn(){}
};

struct Derived1: public virtual Base
{
    void fn(){}
};

struct Derived2: public virtual Base
{
    void fn(){}
};

struct MultiD: public Derived1, public Derived2
{
    void fn(){} // <========== Повинна бути!
};
```

Посилання

- Приведення вверх по успадкуванню – до нащадка, для випадку наявності віртуальних базових класів можливе лише з використанням dynamic_cast<>()[12]
- Для прикладу вище:

```
void fnB(Base& b){ }
void fnDF(DerivedFirst& b){ }
void fnDS(DerivedSecond& b){ }

int main() {
    MultiDerived md;
    fnB(md);
    fnDF(md);
    fnDS(md);

    Base *p = &md;
    fnDF(*dynamic_cast<DerivedFirst*>(p));
    fnDS(*dynamic_cast<DerivedSecond*>(p));
}
```

- Причина – організація розташування членів таких класів у пам'яті.

[12] Та потребуватиме наявності віртуальних функцій в предку – але в реалістичних прикладах вони, зазвичай,

Посилання

- Цікава стаття про реалізацію множинного успадкування в Itanium C++ ABI (Linux, MacOS, gcc/clang, але не MSVC): `https://quuxplusone.github.io/blog/2019/09/30/what-is-the-vtt/`.
- Детальний та простий аналіз, як таблиці віртуальних функцій і суміжна механіка реалізована, на прикладі clang: `https://shaharmike.com/cpp/vtable-part1/`, `https://shaharmike.com/cpp/vtable-part2/`, `https://shaharmike.com/cpp/vtable-part3/`, `https://shaharmike.com/cpp/vtable-part4/`, Переклад перших двох російською: `https://habr.com/ru/company/otus/blog/479802/`.
- `https://quuxplusone.github.io/blog/2024/04/17/name-lookup-in-base-classes/` – "Name lookup in multiple base classes", пояснення, чому коли функція є в двох базових класах, їх не знаходитиме під час перевантаження.

"Девіртуалізація" і std::launder (C++17)

- Приклад проблеми: якщо об'єкт змінив свою природу, наприклад у тій самій пам'яті за допомогою placement new було створено інший об'єкт, хоча вказівники на нього валідні як вказівники на "сиру" пам'ять – послідовність байт, в широкому класі випадків (наприклад, до C++20 – коли об'єкт містить посилання чи константні члени), звертатися за їх допомогою до методів чи полів об'єкту – навіть якщо він того ж типу, буде UB.
- Причина цього UB – оптимізації по девіртуалізації, ранньому зв'язуванню викликів віртуальних функцій, для яких статично відомо тип об'єкту.
- std::launder() вимикає цю оптимізацію і коди типу (взято із стандарту та статті нижче) стає коректним:

```
struct X {      const int n;    };

void test() {
    X *p = new X{1};
    const int a = p->n;
    X *p1 = new (p) X{2};

    const int b = p1->n; // OK
    const int c = p->n;  // UB: p->n може дорівнювати 1.
    const int d = std::launder(p)->n; // OK
}
```

- Детальніше див. https://habr.com/ru/post/540954/ – "C++17. Функция стандартной библиотеки std::launder и задача девиртуализации". Також, там гарно та доступно описано, як працює таблиця віртуальних функцій, зокрема і у випадку множинного успадкування.
- Мушу зізнатися, із опису тут: https://en.cppreference.com/w/cpp/utility/launder я так і не зміг зрозуміти, що це і навіщо...
- Приклад аналізу, коли девіртуалізація відбувається для реальних компіляторів: https://quuxplusone.github.io/blog/2021/02/15/devirtualization/. Зокрема: final допомагає, гарантовано локальна область видимості (включаючи анонімні простори імен) – допомагає.
- Альтернативний підхід до динамічного поліморфізму, популярний у розробці комп'ютерних ігор: https://en.wikipedia.org/wiki/Entity_component_system.

Посилання – патерни та ідіоми

- "Top 7 most used C++ idioms (Part1)" – RAII та Pimpl.
  - "Curiously Recurring Template Pattern (CRTP), Copy-and-swap, Type Erasure".
- "Game++. Patching patterns".
- "Proxy" – альтернативний підхід до динамічного поліморфізму.

